EC-360
SOFTWARE ENGINEERING

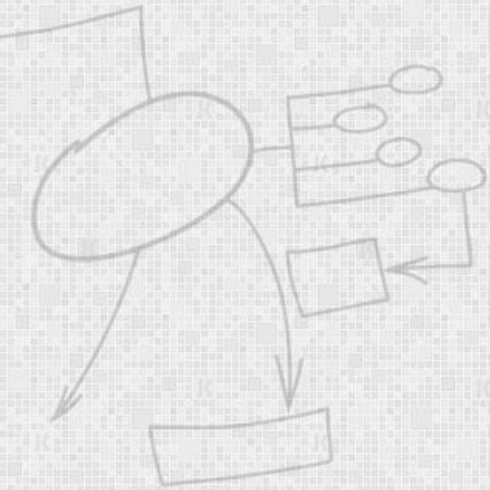
Lecture 4

Software Design Fundamentals

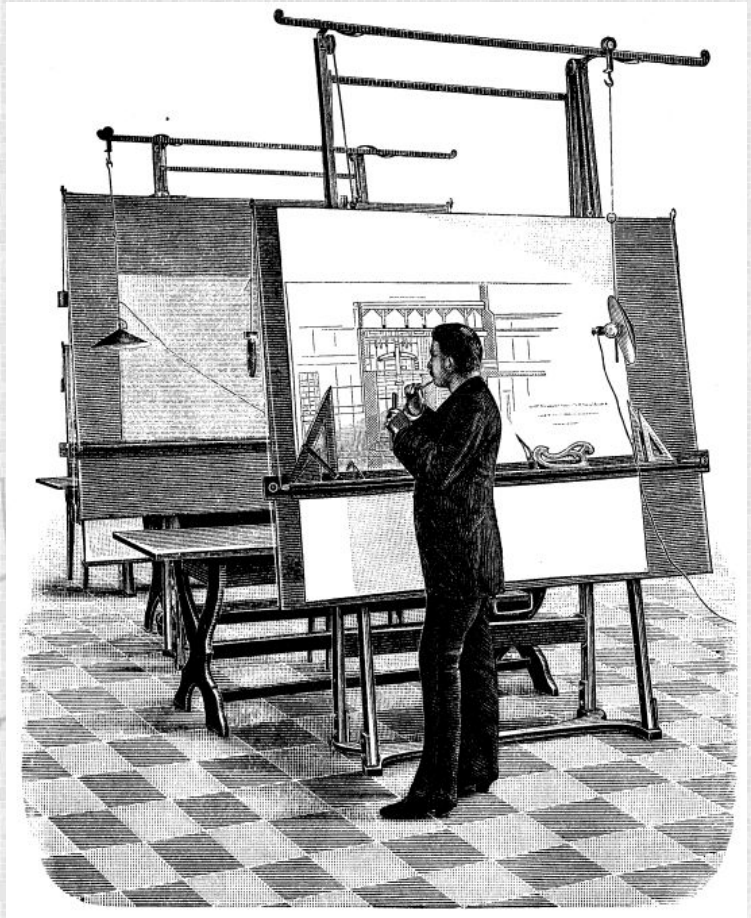
By
Dr Wasi Haider Butt

Today's Lecture: Agenda

- Introduction to software design
- Levels of design
- Coupling and Cohesion



Software Design

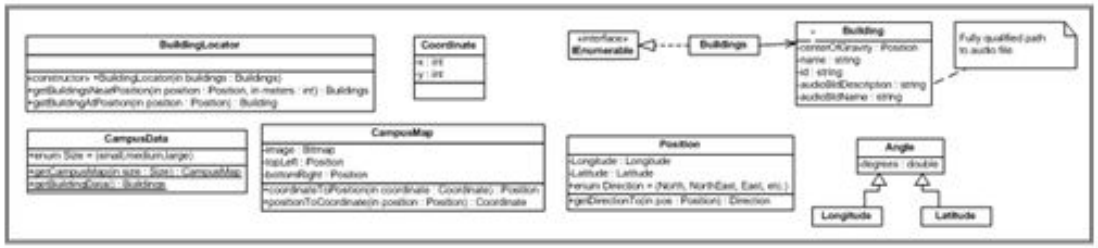


"You can use an eraser on the drafting table or a sledgehammer on the construction site.“

--Frank Lloyd Wright

Opportunities for Design

Product Design
(User Interface)

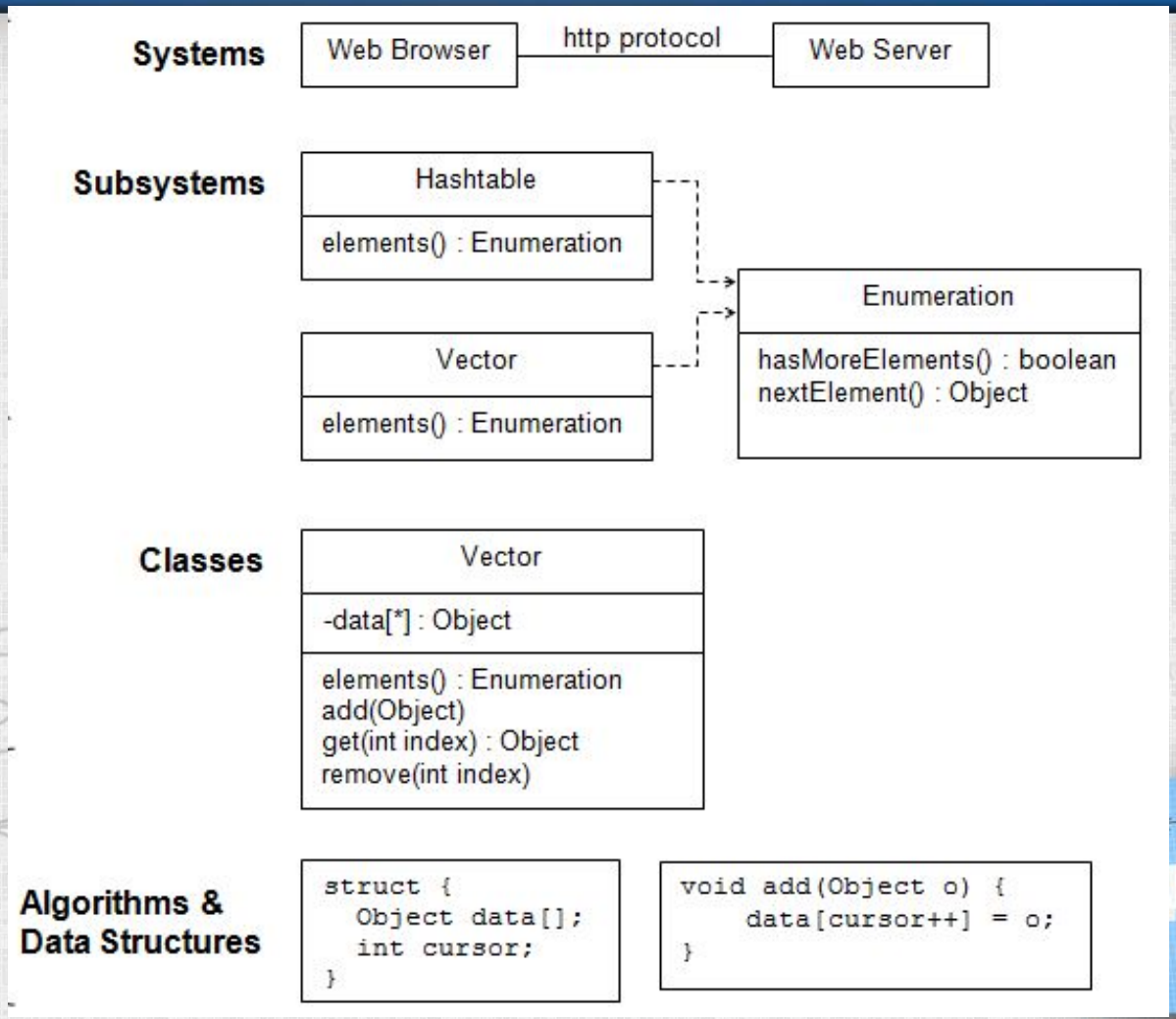


Software Design
(Internal System
Structure)

```
- (void)viewDidAppear:(BOOL)animated {
    [super viewDidAppear:animated];
    // replace the tool bar
    [self.navigationController setToolbarHidden:NO animated:YES];
    modalViewForInstructionsInProcess = NO;
}
```

Code

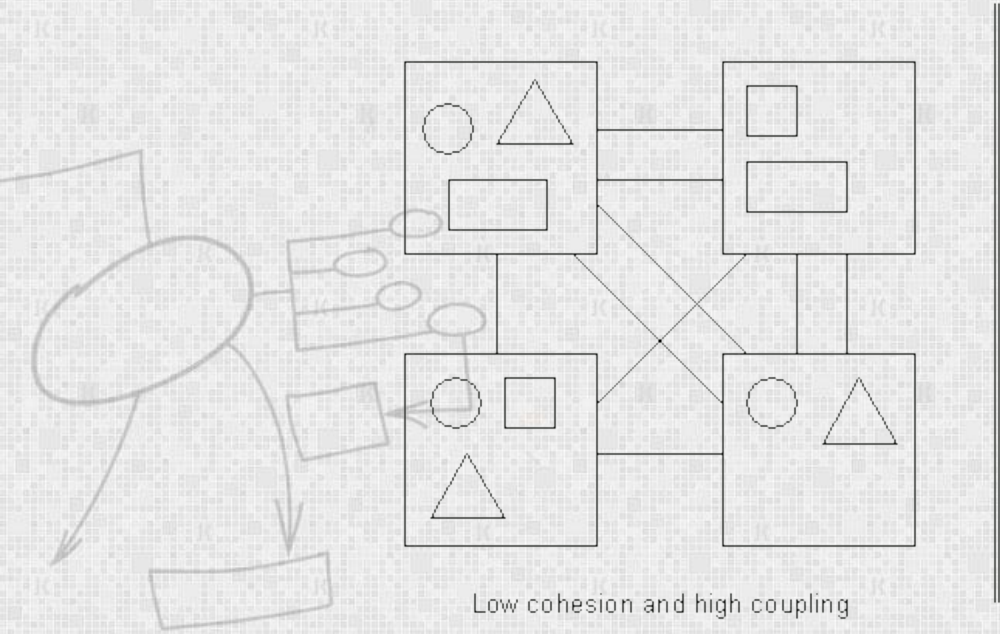
Design Occurs at Different Levels

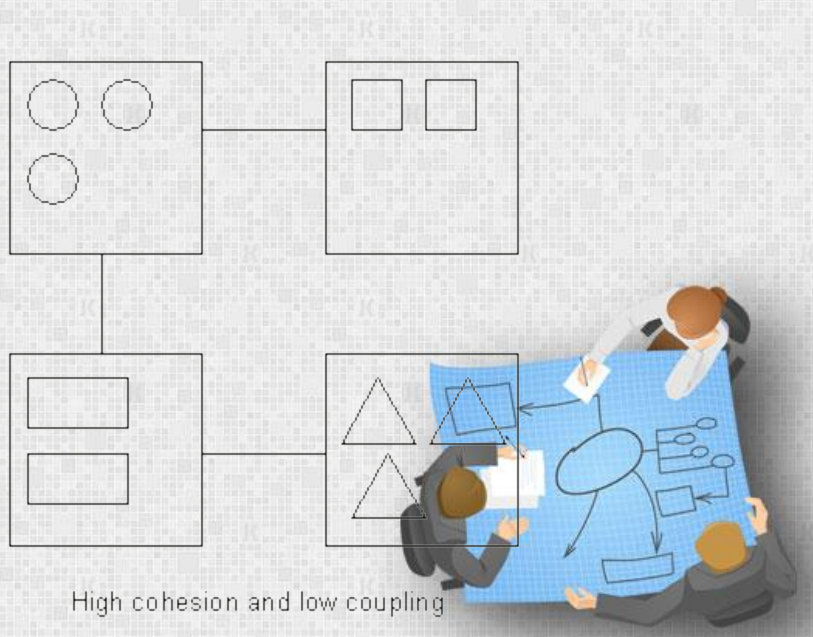


Standard Levels of Design

Cohesion and Coupling

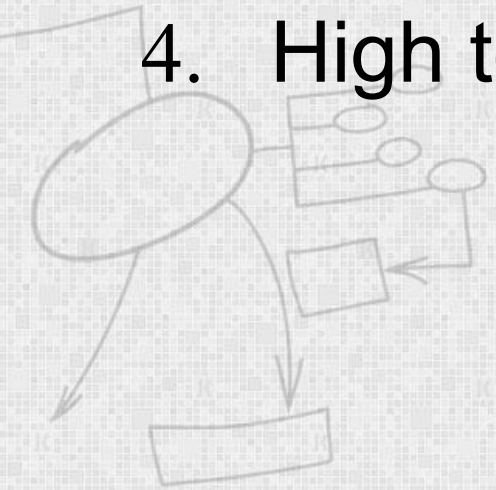
- The best designs have high cohesion (also called strong cohesion) within a module and low coupling (also called weak coupling) between modules.



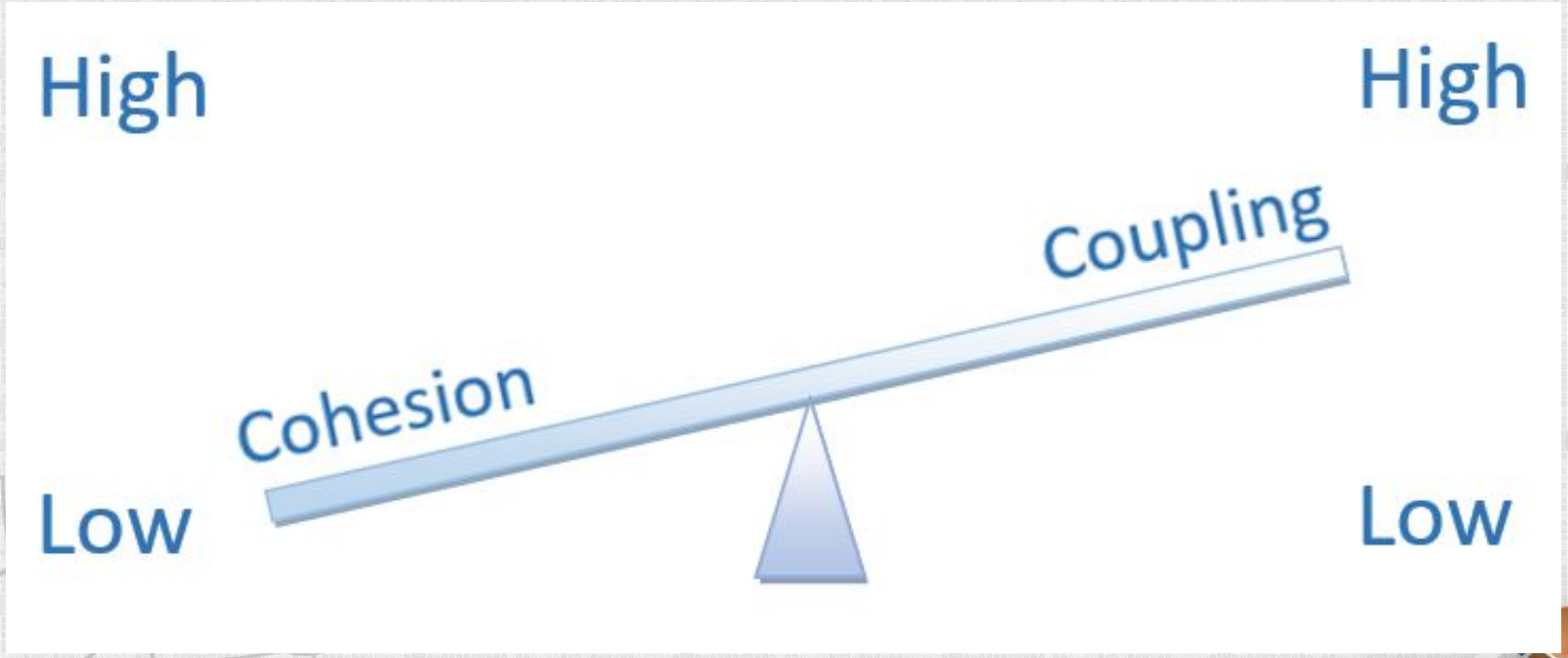


Benefits of high cohesion and low coupling

1. Separation of concerns
2. Fault localization
3. Easy corrective, adaptive and perfective maintenance
4. High testability



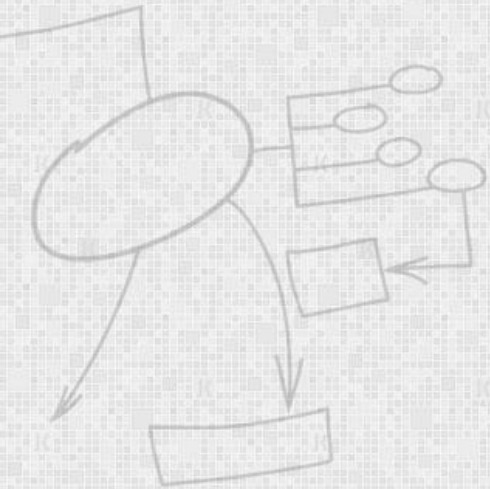
Coupling and Cohesion Tend to be Inversely Correlated



When Coupling is high, cohesion tends to be low and vise versa.

Cohesion

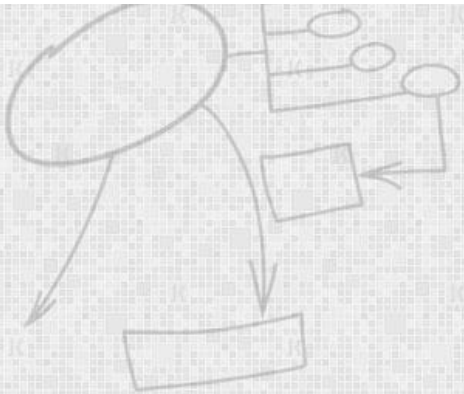
- Cohesion is a measure of functional strength of a module.



Classification of cohesion

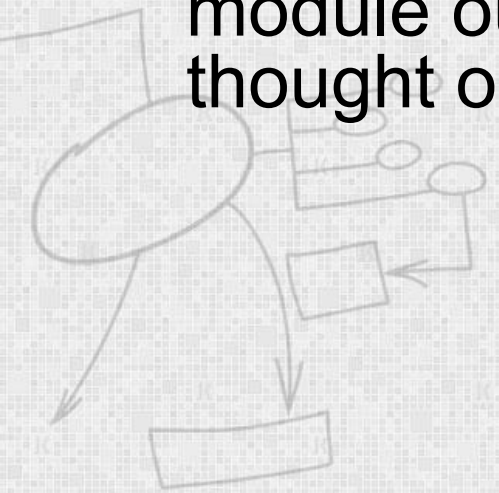
Coincidental	Logical	Temporal	Procedural	Communicational	Sequential	Functional

Low ⟶ High



Coincidental (Random) cohesion:

- A module is said to have coincidental cohesion, if it performs a set of tasks that relate to each other very loosely, if at all.
- It is likely that the functions have been put in the module out of pure coincidence without any thought or design.



Coincidental (Random) cohesion:

1. Fix Car
2. Bake Cake
3. Walk Dog
4. Fill Form
5. Have a Juice
6. Get out of Bed
7. Go the the Movies

Coincidental Cohesion - example

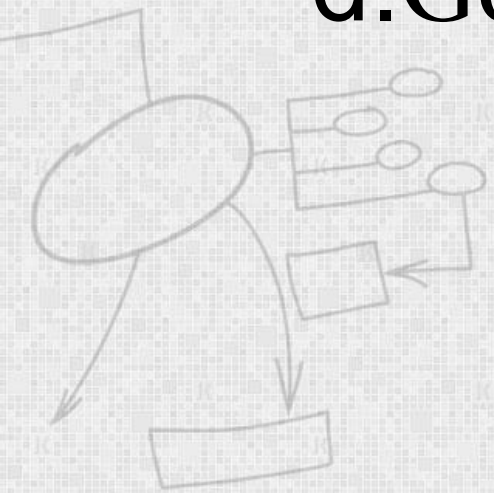
```
/*
  Joe's Stuff
*/
class Joe {
public:
    // converts a path in windows to one in linux
    string win2lin(string);

   // number of days since the beginning of time
   int days(string);

   // outputs a financial report
   void outputreport(financedata, std::ostream&);
};
```

Logical cohesion:

- A module is said to be logically cohesive, if all elements of the module perform similar operations, e.g. error handling, data input, data output, etc
- An example of logical cohesion is the case where a set of print functions generating different output reports are arranged into a single module.

Logical cohesion:

a. Go by Car
b. Go by Train
c. Go by Boat
d. Go by Plane

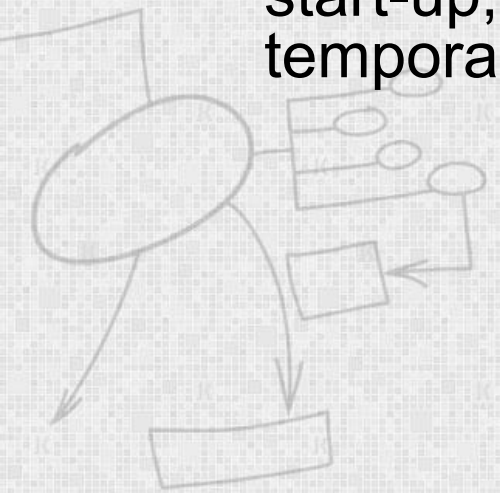


Logical Cohesion

```
class Output {
public:

    // outputs a financial report
    void outputreport(financedata);

   // outputs the current weather
   void outputweather(weatherdata);

   // output a number in a nice formatted way
   void outputint(int);
};
```

Temporal Cohesion

- When a module contains functions that are related by the fact that all the functions must be executed in the same time span, the module is said to exhibit temporal cohesion.
- The set of functions responsible for initialization, start-up, shutdown of some process, etc. exhibit temporal cohesion.



Temporal Cohesion

a. Put out Milk Bottles
b. Put out Cat
c. Turn off TV
d. Brush Teeth

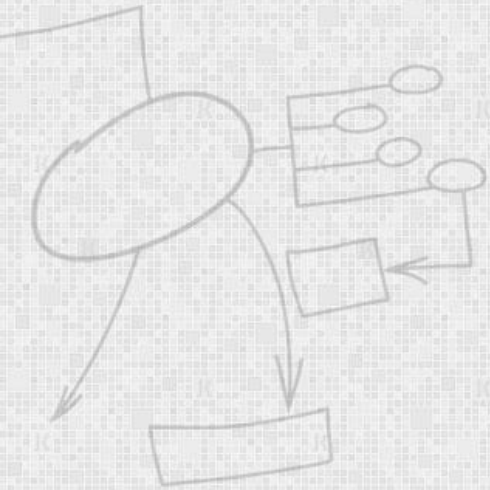
These activities are (only) related by the fact that you do them all late at night, just before you go to bed.

Temporal Cohesion – Example

```
class Init {
public:

    // initializes financial report
    void initreport(financedata);

    // initializes current weather
    void initweather(weatherdata);

    // initializes master count
    void initcount();
}
```

Procedural Cohesion

- A module is said to possess procedural cohesion, if the set of functions of the module are all part of a procedure (algorithm) in which certain sequence of steps have to be carried out for achieving an objective, e.g. the algorithm for decoding a message.



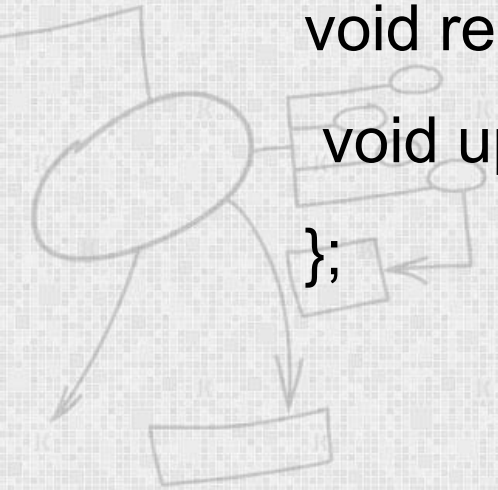
Procedural Cohesion

a. Clean Utensils from Previous Meal
b. Prepare Turkey for Roasting
c. Make Phone Call
d. Take Shower
e. Chop Vegetables
f. Set Table

``Prepare for Holiday Meal:"

Procedural Cohesion – Example

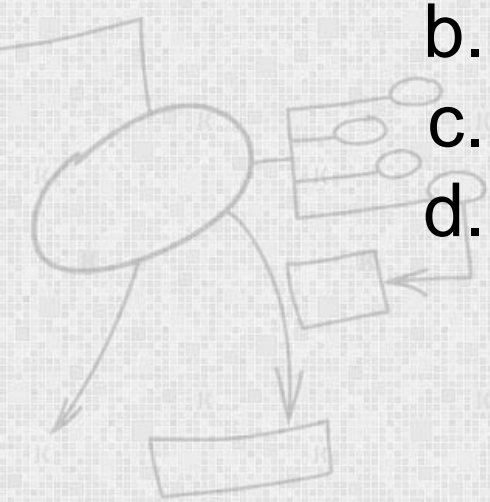
```
class Data {
public:
    // read part number from an input file and update
       //  the directory count
    void read(data&);
     void update (data&);
    };
```



Communicational Cohesion

- A module is said to have communicational cohesion, if all functions of the module refer to or update the same data structure.

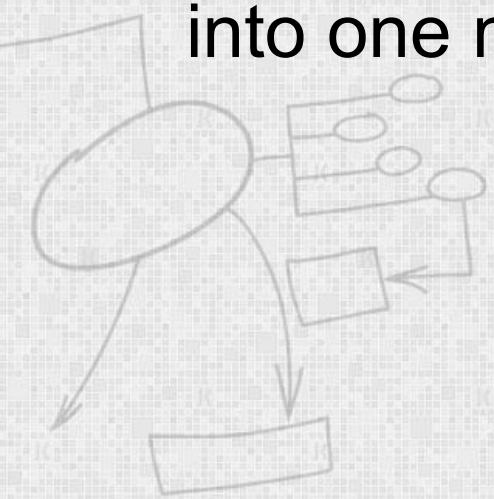
a. Find Title of Book
b. Find Price of Book
c. Find Publisher of Book
d. Find Author of Book



Sequential Cohesion

- A module is said to possess sequential cohesion, if the elements of a module form the parts of sequence, where the output from one element of the sequence is input to the next.

- For example, in a TPS, the get-input, validate-input, sort-input functions are grouped into one module.



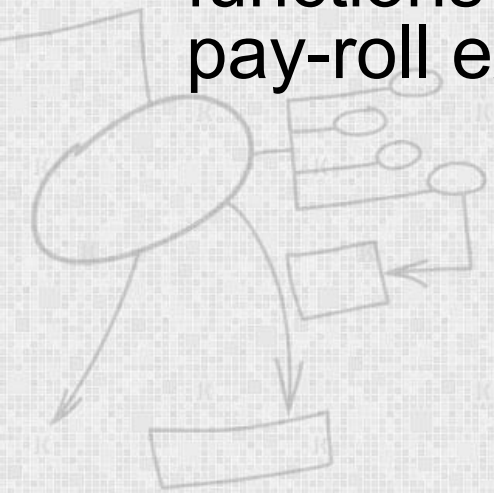
Sequential Cohesion

a. Clean Car Body
b. Fill in Holes in Car
c. Sand Car Body
d. Apply Primer

with the ``car'' being the input that is being ``passed'' as a parameter from task to task.

Functional Cohesion

- Functional cohesion is said to exist, if different elements of a module cooperate to achieve a single function.
- For example, a module containing all the functions required to manage employees' pay-roll exhibits functional cohesion.



Examples of Cohesion-1

Function A	
Function B	Function C
Function D	Function E

Coincidental
Parts unrelated

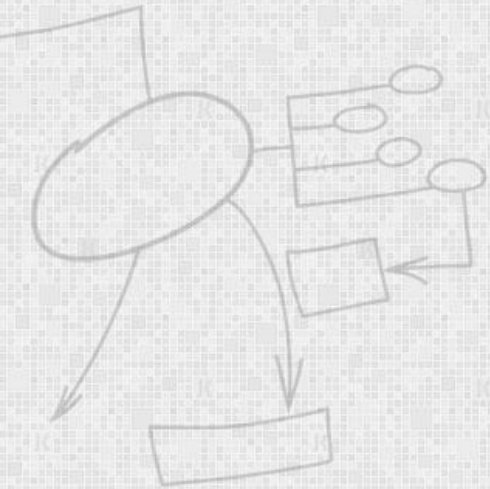
Function A
Function A'
Function A''

logic

Logical
Similar functions

Time t_0
Time t_0 + X
Time t_0 + 2X

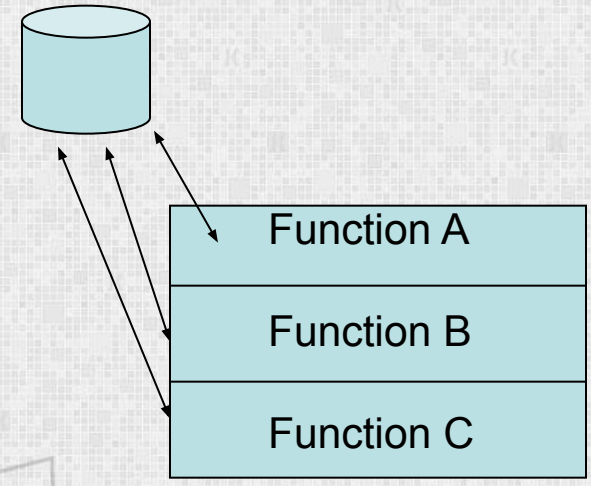
Temporal
Related by time



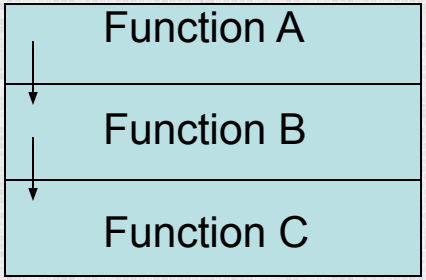
Function A
Function B
Function C

Procedural
Related by order of functions

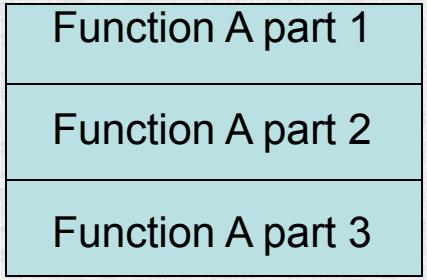
Examples of Cohesion-2



Communicational
Access same data



Sequential
Output of one is input to another

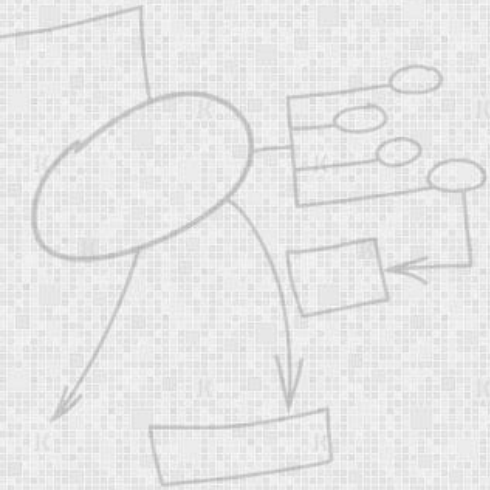


Functional
Sequential with complete, related functions

Coupling

- Coupling between two modules is a measure of the degree of interdependence or interaction between the two modules.

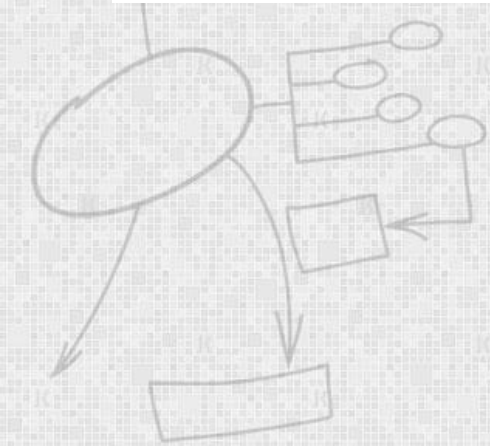
- If two modules interchange large amounts of data, then they are highly interdependent.



Classification of Coupling

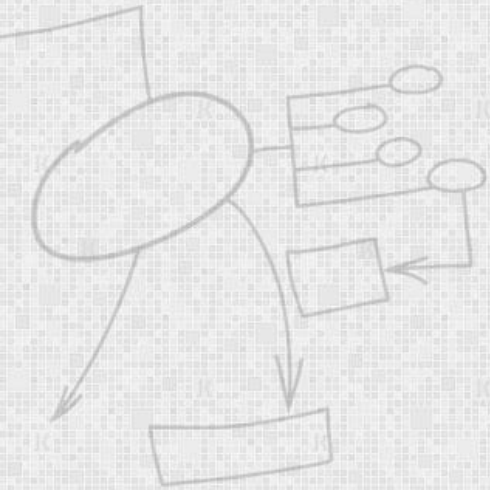
Data	Stamp	Control	Common	Content

Low → High



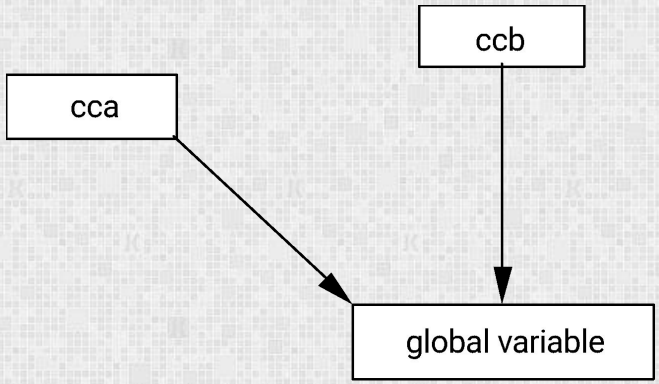
Content Coupling

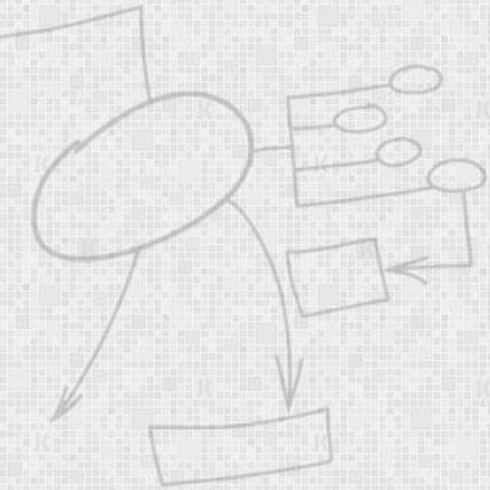
- One module directly references the contents of another
  1. One module modifies the local data or
  2. One module refers to local data in another
  3. One branches to a local label of another



Common Coupling

- Two or more modules connected via global data.
  1. One module writes/updates global data that another module reads





Control Coupling

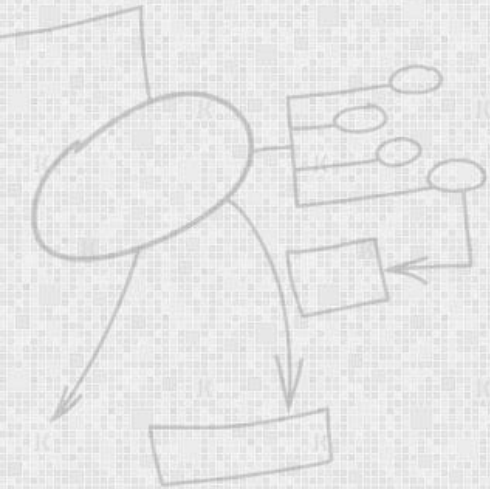
- One module determines the control flow path of another. Example:

```
print(milesTraveled, displayMetricValues)
. . .
public void print(int miles, bool displayMetric) {
    if (displayMetric) {
        System.out.println(. . .);
        . . .
    else { . . .}
}
```

Control Coupling

- One module determines the control flow path of another. Example:

```
print(milesTraveled, displayMetricValues)
. . .
public void print(int miles, bool displayMetric) {
    if (displayMetric) {
        System.out.println(. . .);
        . . .
    else { . . .}
}
```



Stamp Coupling

- Passing a composite data structure to a module that uses only part of it. Example: passing a record with three fields to a module that only needs the first two fields.

- **`calculate withholding (employee record)`**

Data Coupling

- Modules that share data through parameters.

```
Display time of arrival (flight number)
get job with highest priority (job queue)
```